UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SOFTWARE DESIGN AND ARCHITECTURE (SDA)

SESSION 1

Innovating Solutions

01 **INTRODUCTION**

Course Outline and about me.

02 **INTRODUCTION TO SDA**

Hierarchy in design architectural.

03 **SDLC RECAP**

SDLC Process and involvements

04 **SOFTWARE DESIGN**

Challenges and process.

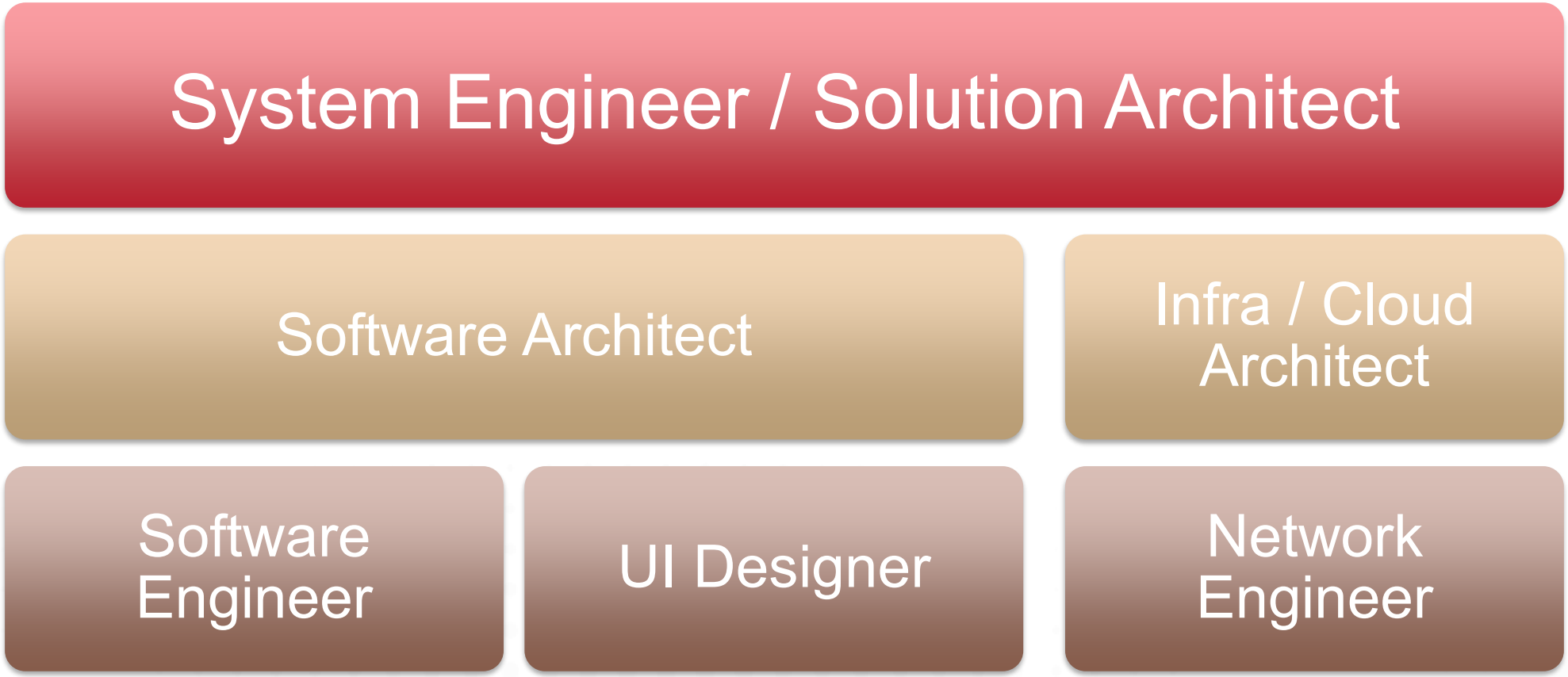
05 **DESIGN FUNDAMENTALS**

Design principle.

06 **PLANNING**

Course planning.

Hierarchy in Software Designers



Their Responsibility?

01

SYSTEMS ENGINEER / SOLUTION ARCHITECT

Designs systems using a holistic approach, which includes designing how software, hardware, people, etc. collaborate to achieve the system's goal.

SOFTWARE ARCHITECT

Design software systems using a black-box modeling approach; concern on the external properties of software components that determine the system's quality and support the further design of functional requirements.

02

03

COMPONENT DESIGNER / SOFTWARE ENGINEER

Focuses on designing the internal structure and behavior of software components identified during the architecture phase; typically, these designers have strong programming skills, since they implement their designs in code.

USER INTERFACE DESIGNER

Design the software's user interface; skilled in determining ways that increase the usability of the system.

04

Software Development Life Cycle

01.Requirement Analysis

Requirement elicitation and analysis

02.Software Design

Architecture, design and tech stacks.

03.Implementation

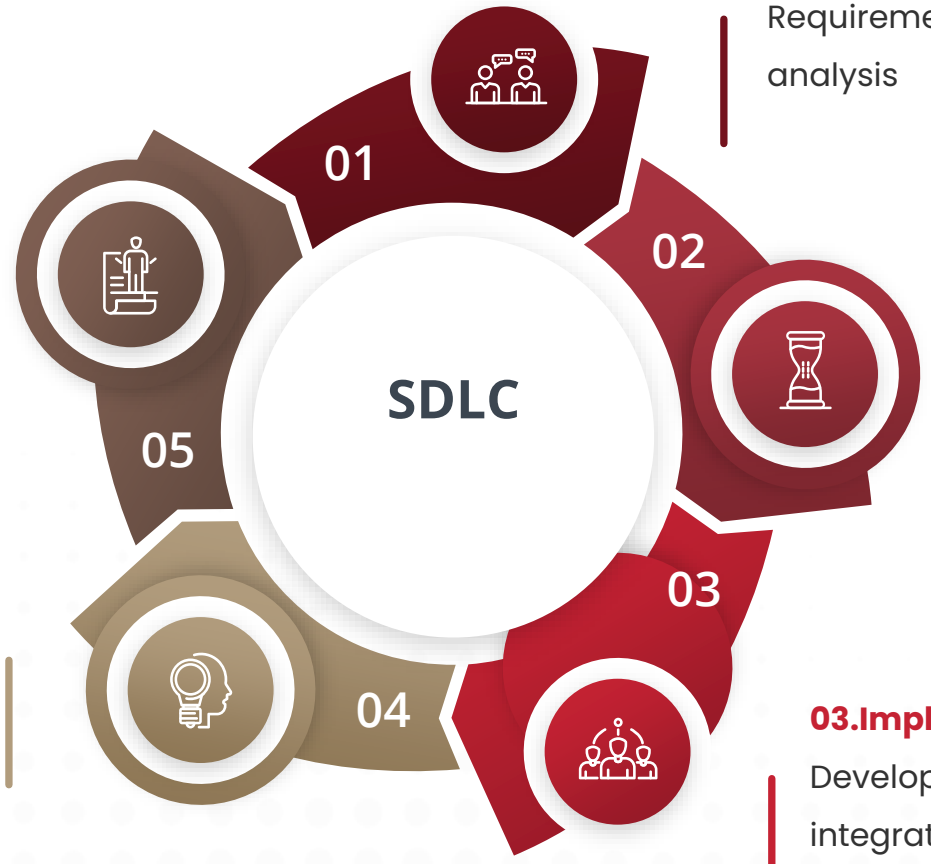
Development, coding and integrations.

04.Testing

Testing and debugging.

05.Deployment and Evolution

Deploy, maintenances and feedbacks.



SDLC Involvement

1	2	3	4	5
Requirement Analysis	Software Design and Architecture	Development and Implementation	Software Testing and Debugging	Deployment and Evolutions
Product Owner Project Manager Business Analyst CTO	Software Architect Solution Architect Software Engineer UI / UX Designer	Front-End Developers Back-End Developers Dev Ops	Solution Architect QA Engineers Testers	Dev Ops Users Support Managers

Common Mistakes to Avoid in SDLC

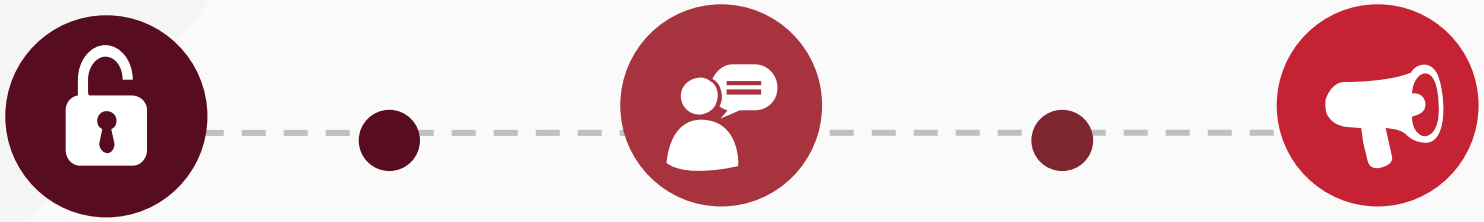


Software Foundation

Why?

To help to accomplish project quickly and efficiently.

To reduce frustrations and confusion along the way.



REQUIREMENTS

Always get verification and validation with stakeholders.

Understanding user needs and project requirements.

DESIGNS

Importances of detailed designs.

Importances of scalability, performances and maintenance.

PROCESS AND DOCUMENTATIONS

Clear and concise process and documentation.

Version control documentation.

Software Design Challenges

01. REQUIREMENTS

Requirement volatility

02. PROCESSES

Inconsistent development processes.

Most company not well define process

03.TECHNOLOGY

Fast, and ever-changing technology.

Co-exist old new tech.

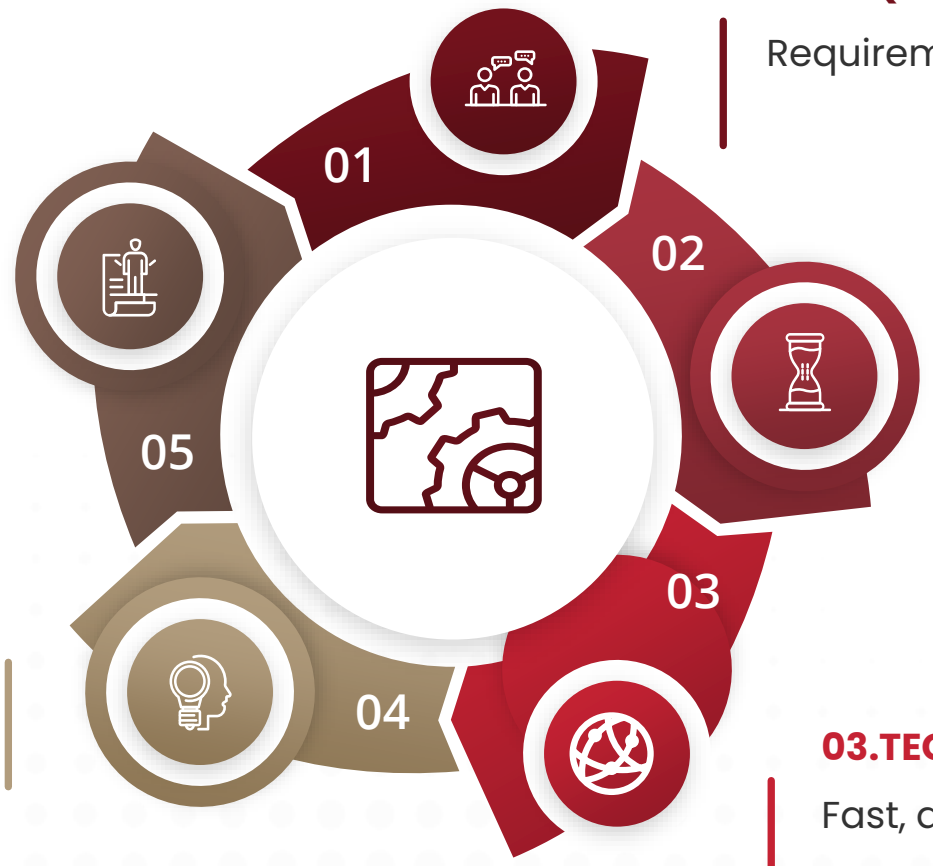
04.ETHICS

Ethical and professional practices.

Tight schedule, negotiation and motivating

05.MANAGEMENT

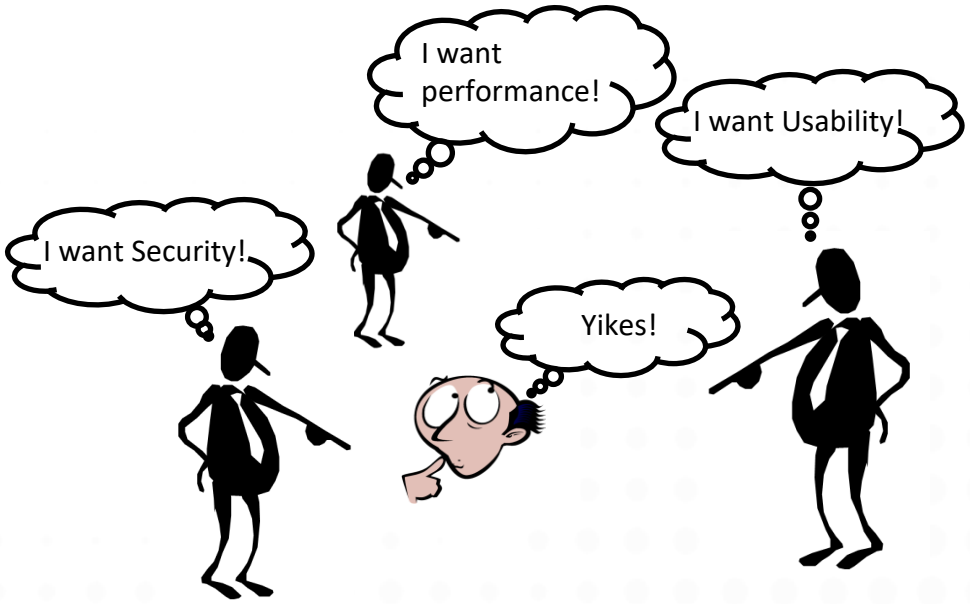
Managing design influences.



Software Design Challenges



Must have a strong understanding of the principles of software design and develop skills to manage complexity and change in software projects



Managing these influences is challenging because it requires designers to span out of the technical domain to have keen interest in the organization as a whole

Software Design Process

Software Architecture

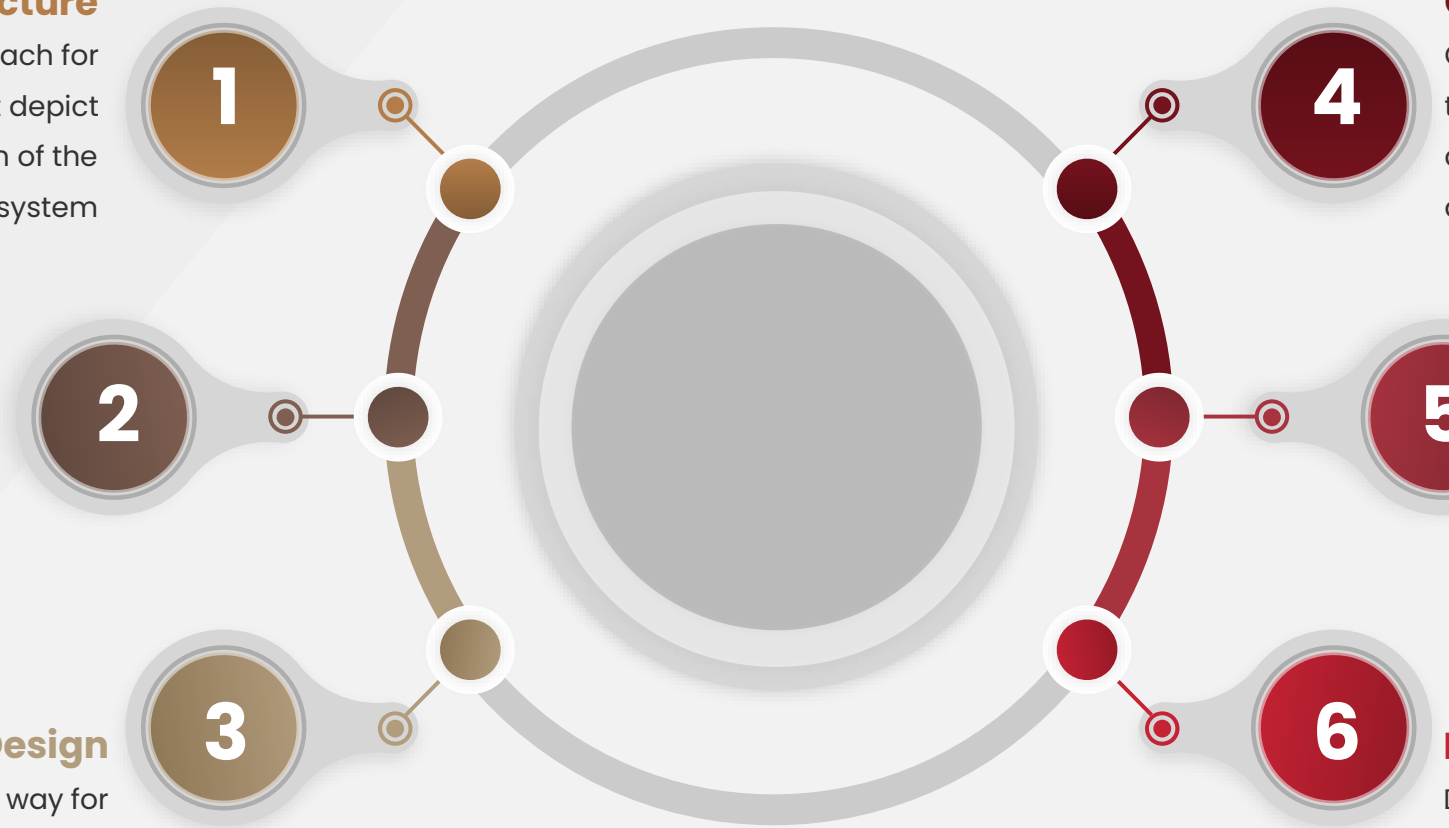
Macro design approach for creating models that depict the quality and function of the software system

Detailed Design

Focuses on functional requirements, whereas the architecture focuses mostly on non-functional, or quality, requirements

Interface Design

Provide a standardized way for accessing services provided by software components



Component Design

Component designs are typically in the form of class diagrams, sequence diagrams, etc

Scoping Design

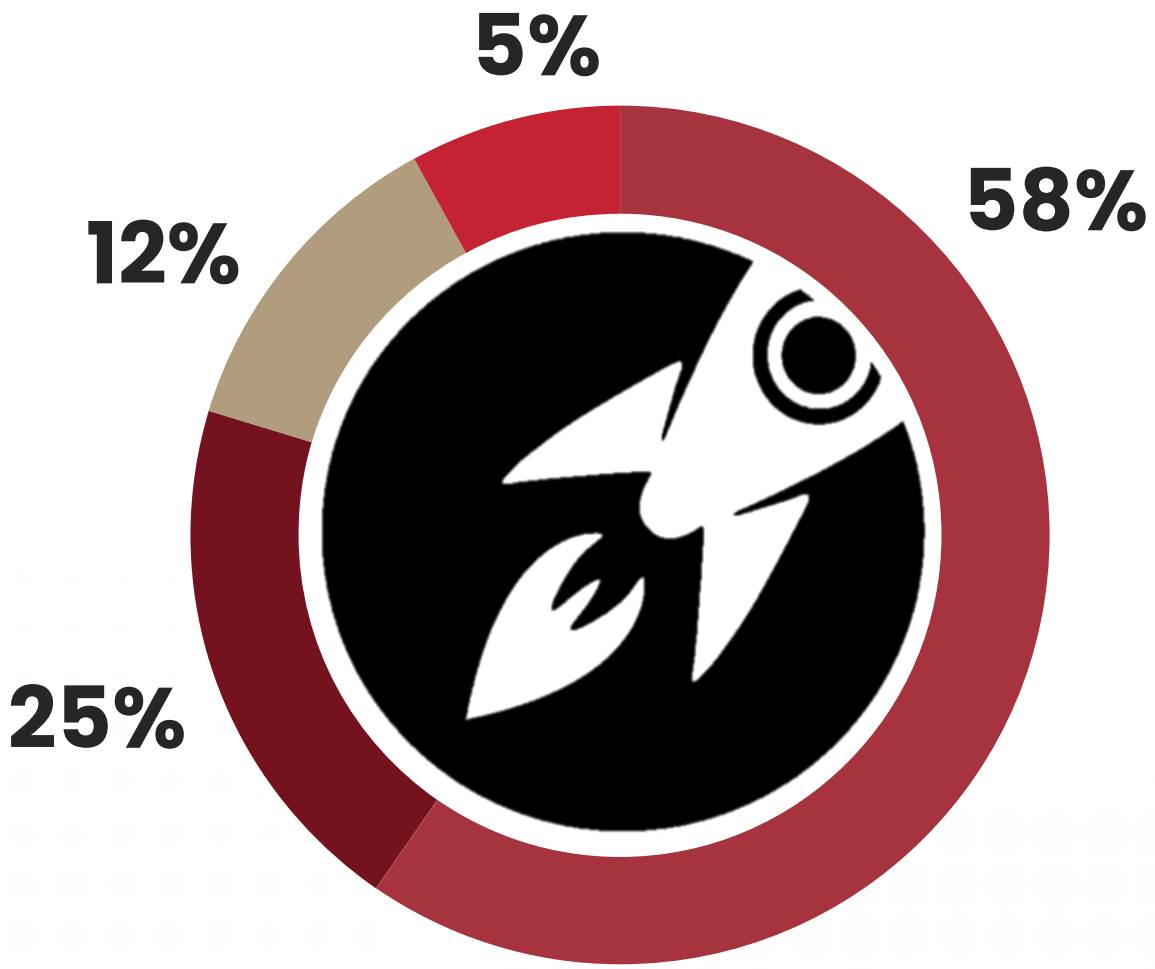
Construction design is important to reason, communicate, and document the solution to complex programming problems

Documentation

Design documentation must be clear and concise. Then submit to be approved.

PLAN AND SCOPE

Define project goal and objectives
Define priority and scopes
Plan the first prototypes or versions of the project (atleast cover the business needs).



58%
Functional

25%
Design

12%
Non-Functional

5%
Sustainability

Why need Roadmap?

01

CLEAR OUT THE OUTLINE PRODUCT OR PROJECT

See all the objective.

Effective scheduling.

MANAGEABLE TIMELINE AND COMMUNICATION **02**

Help ensure to complete on time.

Effective communications.

03

GET EARLY BUY-IN ON DELIVERABLES

Help in getting clients onboard.

Help in getting more interested investors.

MANAGE EXPECTATION STAKEHOLDERS **04**

Avoid from chaos.

As a reminder, everything in on schedule.

Software Design Principle

Modularization

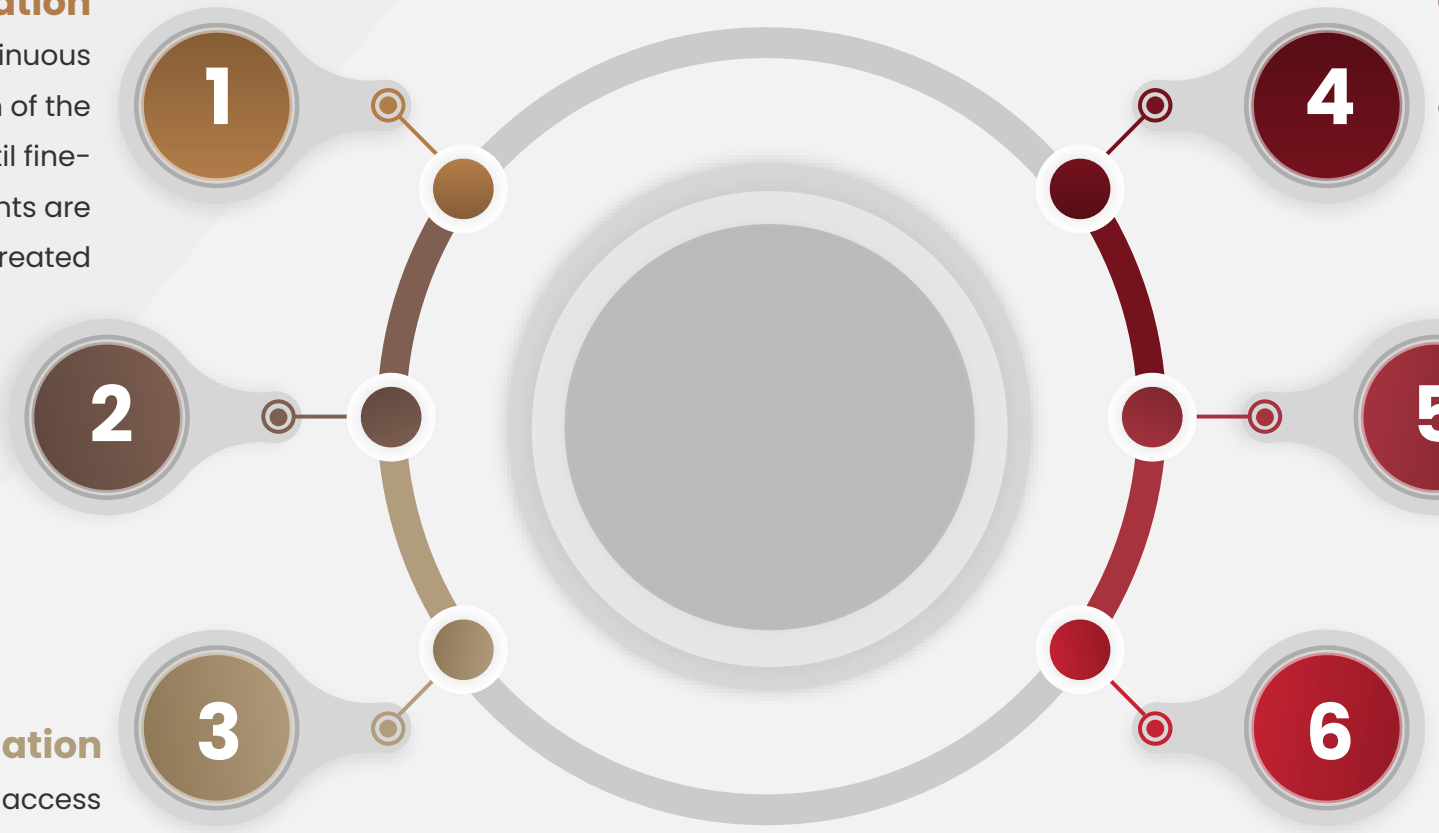
Process of continuous decomposition of the software system until fine-grained components are created

Abstraction

While the principle of modularization specifies what needs to be done, the principle of abstraction provides guidance as to how it should be done

Encapsulation

Encapsulation provides access only to the necessary data of abstracted entities, no more, no less



Coupling

Refers to the manner and degree of interdependence between software modules

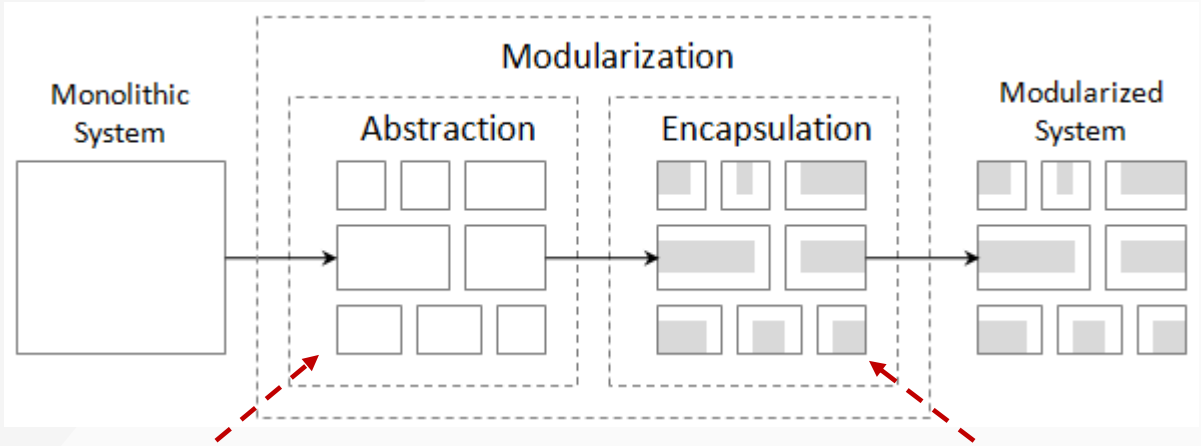
Cohesion

High cohesion good, low cohesion bad

Sufficiency and Completeness

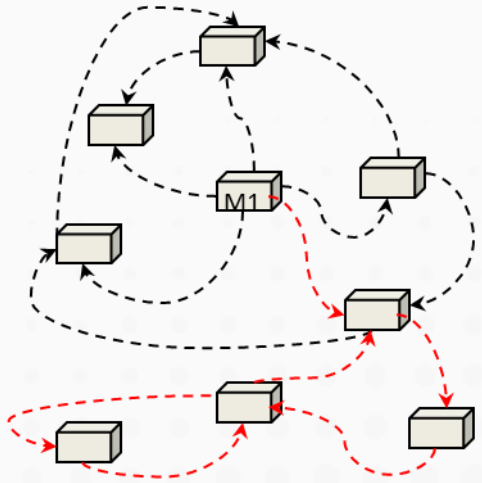
Implies minimal interface and general enough for any prospective client

.

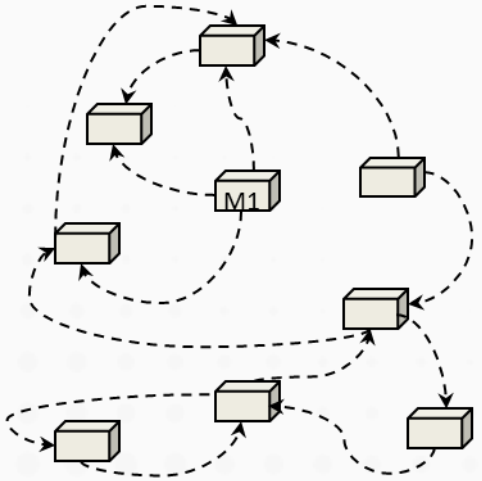


Focus on essential characteristics of entities

Enforce that we only expose essential information



Module 1 (M1) with higher coupling



Module 1 (M1) with lower coupling



Thank you for your participation. I am very grateful to have the opportunity into this sharing session. If there any enquiries, feels free to contact me.

+013 509 9215

khatibsyarbini@utm.my

THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

utm.my univteknologimalaysia utmofficial